

Adam Pinkos
CS4420 - Operating Systems
Assignment 5
October 1st, 2026

Compilation and Execution

The program was written in C using Visual Studio Code and compiled through Ubuntu (WSL) using GCC. I compiled the source code with `gcc scheduler.c -o proj2`. After compiling, I ran the program using the input file and the scheduling algorithm I wanted to test. For Round Robin, I also included the time quantum as the last argument.

```
0      1      2      5      6      7      9      13      14      15      17
pinkos@PinkosDell: /mnt/c/Users/a13pi/Downloads/Fall_2026/CS4420_Operating_Systems/assignment_5$ gcc scheduler.c -o proj2

./proj2 input_1.txt FCFS
./proj2 input_1.txt SJF
./proj2 input_1.txt RR 5

./proj2 input_2.txt FCFS
./proj2 input_2.txt SJF
./proj2 input_2.txt RR 5

./proj2 input_3.txt FCFS
./proj2 input_3.txt SJF
./proj2 input_3.txt RR
```

Test Case 1:

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
pinkos@PinkosDell: /mnt/c/Users/a13pi/Downloads/Fall_2026/CS4420_Operating_Systems/assignment_5$ gcc scheduler.c -o proj2

./proj2 input_1.txt FCFS
./proj2 input_1.txt SJF
./proj2 input_1.txt RR 5
Time 0: Process 0 selected
Time 12: Process 0 finished
Time 12: Process 1 selected
Time 16: Process 1 finished
Time 16: Process 2 selected
Time 17: Process 2 finished
Time 17: Process 3 selected
Time 19: Process 3 finished

Results for each process:
Process 0 | Arrival: 0 | Start: 0 | Running: 12 | End: 12 | Waiting: 0
Process 1 | Arrival: 2 | Start: 12 | Running: 4 | End: 16 | Waiting: 10
Process 2 | Arrival: 3 | Start: 16 | Running: 1 | End: 17 | Waiting: 13
Process 3 | Arrival: 4 | Start: 17 | Running: 2 | End: 19 | Waiting: 13

Average Waiting Time: 9.00

Gantt Chart:
| P0 | P1 | P2 | P3 |
0 12 16 17 19
```

```
Time 0: Process 0 selected
Time 12: Process 0 finished
Time 12: Process 2 selected
Time 13: Process 2 finished
Time 13: Process 3 selected
Time 15: Process 3 finished
Time 15: Process 1 selected
Time 19: Process 1 finished

Results for each process:
Process 0 | Arrival: 0 | Start: 0 | Running: 12 | End: 12 | Waiting: 0
Process 1 | Arrival: 2 | Start: 15 | Running: 4 | End: 19 | Waiting: 13
Process 2 | Arrival: 3 | Start: 12 | Running: 1 | End: 13 | Waiting: 9
Process 3 | Arrival: 4 | Start: 13 | Running: 2 | End: 15 | Waiting: 9

Average Waiting Time: 7.75

Gantt Chart:
| P0 | P2 | P3 | P1 |
0 12 13 15 19
```

```
RR (Time quantum = 5):
Time 0: Process 0 selected
PID 0 ran from 0 to 5 for 5 ms
Time 5: Process 1 selected
PID 1 ran from 5 to 9 for 4 ms
Time 9: Process 1 finished
Time 9: Process 2 selected
PID 2 ran from 9 to 10 for 1 ms
Time 10: Process 2 finished
Time 10: Process 3 selected
PID 3 ran from 10 to 12 for 2 ms
Time 12: Process 3 finished
Time 12: Process 0 selected
PID 0 ran from 12 to 17 for 5 ms
Time 17: Process 0 selected
PID 0 ran from 17 to 19 for 2 ms
Time 19: Process 0 finished

Results for each process:
Process 0 | Arrival: 0 | Start: 0 | Running: 12 | End: 19 | Waiting: 7
Process 1 | Arrival: 2 | Start: 5 | Running: 4 | End: 9 | Waiting: 3
Process 2 | Arrival: 3 | Start: 9 | Running: 1 | End: 10 | Waiting: 6
Process 3 | Arrival: 4 | Start: 10 | Running: 2 | End: 12 | Waiting: 6

Average Waiting Time: 5.50

Gantt Chart:
| P0 | P1 | P2 | P3 | P0 | P0 |
0 5 9 10 12 17 19
```

About Test Case 1:

For the first test, I used the sample input provided in the assignment, which included four processes with different arrival and running times. This was to make sure all three scheduling algorithms were working correctly and producing the expected results. FCFS had an average waiting time of 9.00, SJF had 7.75, and Round Robin with a time quantum of 5 had 5.50. All three matched the professor's expected results, and the Gantt charts show the order in which each process was executed.

Test Case 2:

```
Time 0: Process 0 selected
Time 6: Process 0 finished
Time 6: Process 1 selected
Time 8: Process 1 finished
Time 8: Process 2 selected
Time 12: Process 2 finished
Time 12: Process 3 selected
Time 13: Process 3 finished
Time 13: Process 4 selected
Time 16: Process 4 finished

Results for each process:
Process 0 | Arrival: 0 | Start: 0 | Running: 6 | End: 6 | Waiting: 0
Process 1 | Arrival: 0 | Start: 6 | Running: 2 | End: 8 | Waiting: 6
Process 2 | Arrival: 0 | Start: 8 | Running: 4 | End: 12 | Waiting: 8
Process 3 | Arrival: 0 | Start: 12 | Running: 1 | End: 13 | Waiting: 12
Process 4 | Arrival: 0 | Start: 13 | Running: 3 | End: 16 | Waiting: 13

Average Waiting Time: 7.80
```

Gantt Chart:

P0	P1	P2	P3	P4	
0	6	8	12	13	16

```
Time 0: Process 3 selected
Time 1: Process 3 finished
Time 1: Process 1 selected
Time 3: Process 1 finished
Time 3: Process 4 selected
Time 6: Process 4 finished
Time 6: Process 2 selected
Time 10: Process 2 finished
Time 10: Process 0 selected
Time 16: Process 0 finished
```

```
Results for each process:
Process 0 | Arrival: 0 | Start: 10 | Running: 6 | End: 16 | Waiting: 10
Process 1 | Arrival: 0 | Start: 1 | Running: 2 | End: 3 | Waiting: 1
Process 2 | Arrival: 0 | Start: 6 | Running: 4 | End: 10 | Waiting: 6
Process 3 | Arrival: 0 | Start: 0 | Running: 1 | End: 1 | Waiting: 0
Process 4 | Arrival: 0 | Start: 3 | Running: 3 | End: 6 | Waiting: 3
```

Average Waiting Time: 4.00

Gantt Chart:

P3	P1	P4	P2	P0	
0	1	3	6	10	16

```
RR (Time quantum = 5):
Time 0: Process 0 selected
PID 0 ran from 0 to 5 for 5 ms
Time 5: Process 1 selected
PID 1 ran from 5 to 7 for 2 ms
Time 7: Process 1 finished
Time 7: Process 2 selected
PID 2 ran from 7 to 11 for 4 ms
Time 11: Process 2 finished
Time 11: Process 3 selected
PID 3 ran from 11 to 12 for 1 ms
Time 12: Process 3 finished
Time 12: Process 4 selected
PID 4 ran from 12 to 15 for 3 ms
Time 15: Process 4 finished
Time 15: Process 0 selected
PID 0 ran from 15 to 16 for 1 ms
Time 16: Process 0 finished
```

```
Results for each process:
Process 0 | Arrival: 0 | Start: 0 | Running: 6 | End: 16 | Waiting: 10
Process 1 | Arrival: 0 | Start: 5 | Running: 2 | End: 7 | Waiting: 5
Process 2 | Arrival: 0 | Start: 7 | Running: 4 | End: 11 | Waiting: 7
```

About Test Case 2:

For this test, I used five processes that all arrived at time 0 but had different running times. This was to see how each scheduling algorithm handled having all processes ready at once. FCFS completed them in the order they were listed, giving an average waiting time of 7.80. SJF selected the shortest processes first and had an average waiting time of 4.00. Round Robin used a time quantum of 5 and had an average waiting time of 9.00. The Gantt charts show the different execution orders for each algorithm.

Test Case 3:

```
./proj2 input_3.txt FCFS
./proj2 input_3.txt SJF
./proj2 input_3.txt RR 5
Time 0: idle
Time 1: idle
Time 2: Process 0 selected
Time 5: Process 0 finished
Time 5: idle
Time 6: idle
Time 7: Process 1 selected
Time 9: Process 1 finished
Time 9: Process 2 selected
Time 13: Process 2 finished
Time 13: idle
Time 14: idle
Time 15: Process 3 selected
Time 17: Process 3 finished

Results for each process:
Process 0 | Arrival: 2 | Start: 2 | Running: 3 | End: 5 | Waiting: 0
Process 1 | Arrival: 7 | Start: 7 | Running: 2 | End: 9 | Waiting: 0
Process 2 | Arrival: 8 | Start: 9 | Running: 4 | End: 13 | Waiting: 1
Process 3 | Arrival: 15 | Start: 15 | Running: 2 | End: 17 | Waiting: 0

Average Waiting Time: 0.25

Gantt Chart:
| IDLE | IDLE | P0 | IDLE | IDLE | P1 | P2 | IDLE | IDLE | P3 |
0      1      2      5      6      7      9      13     14     15     17
```

```
Time 0: idle
Time 1: idle
Time 2: Process 0 selected
Time 5: Process 0 finished
Time 5: idle
Time 6: idle
Time 7: Process 1 selected
Time 9: Process 1 finished
Time 9: Process 2 selected
Time 13: Process 2 finished
Time 13: idle
Time 14: idle
Time 15: Process 3 selected
Time 17: Process 3 finished

Results for each process:
Process 0 | Arrival: 2 | Start: 2 | Running: 3 | End: 5 | Waiting: 0
Process 1 | Arrival: 7 | Start: 7 | Running: 2 | End: 9 | Waiting: 0
Process 2 | Arrival: 8 | Start: 9 | Running: 4 | End: 13 | Waiting: 1
Process 3 | Arrival: 15 | Start: 15 | Running: 2 | End: 17 | Waiting: 0

Average Waiting Time: 0.25

Gantt Chart:
| IDLE | IDLE | P0 | IDLE | IDLE | P1 | P2 | IDLE | IDLE | P3 |
0      1      2      5      6      7      9      13     14     15     17
```

```
RR (Time quantum = 5):
Time 0: idle
Time 1: idle
Time 2: Process 0 selected
PID 0 ran from 2 to 5 for 3 ms
Time 5: Process 0 finished
Time 5: idle
Time 6: idle
Time 7: Process 1 selected
PID 1 ran from 7 to 9 for 2 ms
Time 9: Process 1 finished
Time 9: Process 2 selected
PID 2 ran from 9 to 13 for 4 ms
Time 13: Process 2 finished
Time 13: idle
Time 14: idle
Time 15: Process 3 selected
PID 3 ran from 15 to 17 for 2 ms
Time 17: Process 3 finished

Results for each process:
Process 0 | Arrival: 2 | Start: 2 | Running: 3 | End: 5 | Waiting: 0
Process 1 | Arrival: 7 | Start: 7 | Running: 2 | End: 9 | Waiting: 0
Process 2 | Arrival: 8 | Start: 9 | Running: 4 | End: 13 | Waiting: 1
Process 3 | Arrival: 15 | Start: 15 | Running: 2 | End: 17 | Waiting: 0
```

About Test Case 3:

For the third test, I used four processes with different arrival times and gaps between them. This was to check if the program correctly displayed when the CPU was idle and continued running once another process arrived. The screenshots show the idle periods, along with the start, end, and waiting times for each process. The average waiting time was 0.25, showing that most processes were able to run shortly after arriving.

